

EXPERIMENT – 3

AIM: To design and implement an Employee–Department–Project database schema and perform SQL queries using Selection, Projection, Aggregate Functions, GROUP BY, HAVING, CASE expressions, and ORDER BY.

SOURCE CODE:

DATABASE CREATING

```
CREATE DATABASE EmployeeManagement;  
USE EmployeeManagement;
```

TABLE CREATING

```
CREATE TABLE Department (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(50) NOT NULL,  
    location VARCHAR(50) );  
  
CREATE TABLE Employee (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50) NOT NULL,  
    gender CHAR(1), salary DECIMAL(10,2),  
    hire_date DATE,  
    dept_id INT,  
    FOREIGN KEY (dept_id)  
        REFERENCES Department(dept_id)  
);  
  
CREATE TABLE Project (  
    project_id INT PRIMARY KEY,  
    project_name VARCHAR(100) NOT NULL,  
    budget DECIMAL(12,2),  
    dept_id INT,  
    FOREIGN KEY (dept_id)  
        REFERENCES Department(dept_id)  
);  
  
CREATE TABLE Employee_Project (  
    emp_id INT,  
    project_id INT,  
    hours_worked INT,  
    PRIMARY KEY (emp_id, project_id),  
    FOREIGN KEY (emp_id)  
        REFERENCES Employee(emp_id),  
    FOREIGN KEY (project_id)  
        REFERENCES Project(project_id)  
);
```

INSERTING DEPARTMENTS

```
INSERT INTO Department (dept_id, dept_name, location)
VALUES
(1, 'Development', 'Delhi'),
(2, 'Human Resources', 'Mumbai'),
(3, 'Accounting', 'Bangalore'),
(4, 'Sales', 'Pune'),
(5, 'Innovation', 'Hyderabad');
```

INSERTING 30 EMPLOYEES

```
INSERT INTO Employee
(emp_id, emp_name, gender, salary, hire_date, dept_id)
VALUES
(101, 'Aarush Khanna', 'M', 78000, '2021-01-15', 1),
(102, 'Ishita Roy', 'F', 69000, '2022-03-10', 1),
(103, 'Vivaan Sinha', 'M', 84000, '2020-07-20', 1),
(104, 'Myra Joshi', 'F', 74000, '2021-11-05', 1),
(105, 'Reyansh Kapoor', 'M', 97000, '2019-06-18', 1),
(106, 'Anika Bhatia', 'F', 66000, '2023-01-12', 1),
(107, 'Kabir Malhotra', 'M', 57000, '2022-05-15', 2),
(108, 'Aadhya Menon', 'F', 64000, '2021-09-22', 2),
(109, 'Vihaan Arora', 'M', 60000, '2020-04-11', 2),
(110, 'Kiara Saxena', 'F', 69000, '2023-02-19', 2),
(111, 'Advait Rao', 'M', 62000, '2022-08-01', 2),
(112, 'Saanvi Desai', 'F', 73000, '2019-12-10', 2),
(113, 'Dhruv Mehta', 'M', 90000, '2020-01-25', 3),
(114, 'Navya Pillai', 'F', 94000, '2019-08-14', 3),
(115, 'Arnav Kulkarni', 'M', 78000, '2021-06-30', 3),
(116, 'Tara Chawla', 'F', 87000, '2022-10-17', 3),
(117, 'Ishaan Nair', 'M', 75000, '2023-03-21', 3),
(118, 'Riddhi Shah', 'F', 92000, '2020-11-09', 3),
(119, 'Krish Bansal', 'M', 72000, '2021-02-28', 4),
(120, 'Avni Khurana', 'F', 68000, '2022-04-15', 4),
(121, 'Rudra Mishra', 'M', 76000, '2020-09-12', 4),
(122, 'Pari Anand', 'F', 83000, '2019-07-07', 4),
(123, 'Yuvaan Sethi', 'M', 71000, '2023-01-05', 4),
(124, 'Diya Oberoi', 'F', 79000, '2021-12-18', 4),
(125, 'Atharv Reddy', 'M', 99000, '2018-05-20', 5),
(126, 'Meher Kaur', 'F', 93000, '2020-03-16', 5),
(127, 'Shaurya Das', 'M', 89000, '2021-10-08', 5),
(128, 'Zoya Fernandes', 'F', 95000, '2019-11-25', 5),
(129, 'Ayaan Thakur', 'M', 81000, '2022-07-14', 5),
(130, 'Vanya Kapoor', 'F', 98000, '2023-02-02', 5);
```

INSERTING 8 PROJECTS

```
INSERT INTO Project (project_id, project_name, budget, dept_id) VALUES
(201, 'Smart Web Portal', 550000, 1),
(202, 'Cloud Infrastructure', 800000, 1),
(203, 'Staff Development Program', 300000, 2),
(204, 'Budget Forecasting System', 650000, 3),
(205, 'Online Sales Campaign', 480000, 4),
(206, 'Customer Insights Study', 400000, 4),
(207, 'Predictive Analytics Platform', 950000, 5),
(208, 'Cybersecurity Framework', 1100000, 5);
```

ASSIGNING EMPLOYEES TO PROJECTS

```
INSERT INTO Employee_Project (emp_id, project_id, hours_worked)
VALUES
(101, 201, 120),
(102, 201, 100),
(103, 202, 150),
(104, 202, 110),
(105, 201, 130),
(106, 202, 90),
(107, 203, 100),
(108, 203, 120),
(109, 203, 80),
(110, 203, 110),
(111, 203, 90),
(112, 203, 130),
(113, 204, 140),
(114, 204, 150),
(115, 204, 120),
(116, 204, 135),
(117, 204, 100),
(118, 204, 145),
(119, 205, 110),
(120, 205, 100),
(121, 206, 120),
(122, 205, 140),
(123, 206, 90),
(124, 206, 130),
(125, 207, 160),
(126, 207, 140),
(127, 208, 150),
(128, 208, 160),
(129, 207, 120),
(130, 208, 135);
```

SQL QUERIES:

SELECTION

SELECT

e.employee_id,
e.first_name,
e.last_name,
s.amount AS salary

FROM employees e

JOIN salaries s

ON e.employee_id = s.employee_id

WHERE s.amount > 80000;

The screenshot shows the MySQL Workbench interface. On the left is the 'EXPLORER' pane with a tree view of database objects. The 'TABLES' section is expanded, showing a list of tables including 'course', 'department', 'department_salary_summary', 'departments', 'employee_basic', 'employee_hierarchy', 'employees', 'enrollment', 'faculty', 'laves', 'salaries', 'student', 'student_backup', and 'teachers'. The 'FUNCTIONS' section includes 'add_numbers', 'annual_salary', 'calculate_grade', 'getAverageSalaryOfDepartment', 'square_number', and 'student_result'. The 'PROCEDURES' section includes 'AddStudent', 'ApproveOrRejectLeave', 'GetAllEmployees', 'GetAllStudents', 'GetEmployeesByDepartment', and 'GetStudentsByDepartment'. The main area displays 'Query 1' with the following SQL code:

```
1 SHOW TABLES;
2
3 SELECT
4   e.employee_id,
5   e.first_name,
6   e.last_name,
7   s.amount AS salary
8 FROM employees e
9 JOIN salaries s
10  ON e.employee_id = s.employee_id
11 WHERE s.amount > 80000;
```

Below the query, the results are shown in a table with the following data:

employee_id	first_name	last_name	salary
3	Robert	Johnson	90000
4	Amanda	Williams	100000
5	Michael	Brown	85000
6	Sara	Johnson	95000
7	Daniel	Miller	110000
8	Megan	Clark	88000
9	Christopher	Harris	92000
11	Elena	Martinez	87000
13	Maria	Garcia	92000
14	Tariq	Ali	105000
15	Sofia	Lopez	90000
16	Jamal	Williams	95000
17	Isabella	Davis	110000
18	Naveed	Choudhry	87000
19	Aisha	Malik	92000

PROJECTION

SELECT

CONCAT(e.first_name, ' ', e.last_name) AS emp_name,
s.amount AS salary

FROM employees e

JOIN salaries s

ON e.employee_id = s.employee_id;

The screenshot shows the MySQL Workbench interface. On the left is the 'EXPLORER' pane with a tree view of database objects. The 'TABLES' section is expanded, showing a list of tables including 'course', 'department', 'department_salary_summary', 'departments', 'employee_basic', 'employee_hierarchy', 'employees', 'enrollment', 'faculty', 'laves', 'salaries', 'student', 'student_backup', and 'teachers'. The 'FUNCTIONS' section includes 'add_numbers', 'annual_salary', 'calculate_grade', 'getAverageSalaryOfDepartment', 'square_number', and 'student_result'. The 'PROCEDURES' section includes 'AddStudent', 'ApproveOrRejectLeave', 'GetAllEmployees', 'GetAllStudents', 'GetEmployeesByDepartment', and 'GetStudentsByDepartment'. The main area displays 'Query 1' with the following SQL code:

```
12
13 SELECT
14   CONCAT(e.first_name, ' ', e.last_name) AS emp_name,
15   s.amount AS salary
16 FROM employees e
17 JOIN salaries s
18  ON e.employee_id = s.employee_id;
```

Below the query, the results are shown in a table with the following data:

emp_name	salary
John Doe	80000
Jane Smith	75000
Robert Johnson	90000
Amanda Williams	100000
Michael Brown	85000
Sara Johnson	95000
Daniel Miller	110000
Megan Clark	88000
Christopher Harris	92000
Olivia White	80000
Elena Martinez	87000
Ahmed Khan	78000
Maria Garcia	92000
Tariq Ali	105000
Sofia Lopez	90000
Jamal Williams	95000
Isabella Davis	110000
Naveed Choudhry	87000
Aisha Malik	92000

AGGREGATE FUNCTIONS

SELECT

```
COUNT(DISTINCT employee_id) AS total_employees,  
SUM(amount) AS total_salary,  
AVG(amount) AS average_salary,  
MAX(amount) AS highest_salary,  
MIN(amount) AS lowest_salary
```

FROM salaries;



The screenshot shows a MySQL IDE window titled 'MySQL - db_44yurwregj'. The 'EXPLORER' panel on the left lists tables including 'course', 'department', 'department_salary_summary', 'departments', 'employee_basic', 'employee_hierarchy', and 'employees'. The 'Query 1' tab is active, displaying the following SQL query:

```
21  
22 SELECT  
23     COUNT(DISTINCT employee_id) AS total_employees,  
24     SUM(amount) AS total_salary,  
25     AVG(amount) AS average_salary,  
26     MAX(amount) AS highest_salary,  
27     MIN(amount) AS lowest_salary  
28 FROM salaries;
```

The results are shown in a table with the following data:

total_employees	total_salary	average_salary	highest_salary	lowest_salary
20	1811000	90550.0000	110000	75000

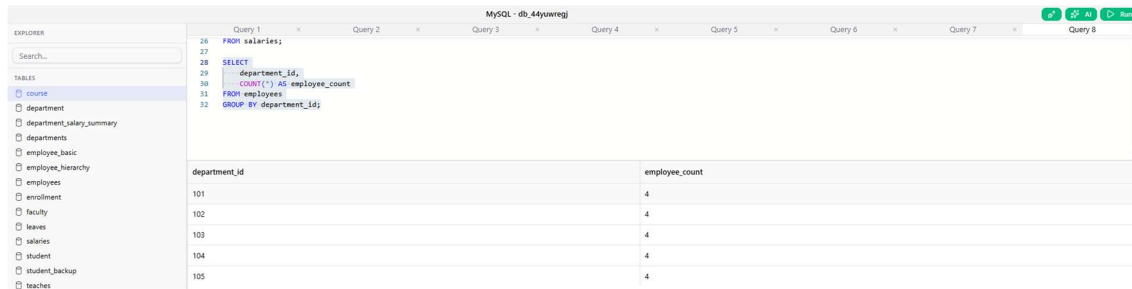
GROUP BY

SELECT

```
dept_id,  
COUNT(*) AS employee_count
```

FROM Employee

GROUP BY dept_id;



The screenshot shows a MySQL IDE window titled 'MySQL - db_44yurwregj'. The 'EXPLORER' panel on the left lists tables including 'course', 'department', 'department_salary_summary', 'departments', 'employee_basic', 'employee_hierarchy', 'employees', 'enrollment', 'faculty', 'leaves', 'salaries', 'student', 'student_backup', and 'teaches'. The 'Query 1' tab is active, displaying the following SQL query:

```
26 FROM salaries;  
27  
28 SELECT  
29     department_id,  
30     COUNT(*) AS employee_count  
31 FROM employees  
32 GROUP BY department_id;
```

The results are shown in a table with the following data:

department_id	employee_count
101	4
102	4
103	4
104	4
105	4

GROUP BY WITH JOIN

SELECT

```
d.department_name,  
AVG(s.amount) AS average_salary
```

FROM departments d

JOIN employees e

```
ON d.department_id = e.department_id
```

JOIN salaries s

```
ON e.employee_id = s.employee_id
```

GROUP BY d.department_name;

MySQL - db_44yowregj

Query 1 x Query 2 x Query 3 x Query 4 x Query 5 x Query 6 x Query 7 x Query 8 x

```

34 SELECT
35     d.department_name,
36     AVG(s.amount) AS average_salary
37 FROM departments d
38 JOIN employees e
39     ON d.department_id = e.department_id
40 JOIN salaries s
41     ON e.employee_id = s.employee_id
42 GROUP BY d.department_name;

```

department_name	average_salary
Engineering	89250.0000
Analytics	93250.0000
Design	89250.0000
Management	97250.0000
Marketing	83750.0000

HAVING

SELECT

d.department_name,
AVG(s.amount) AS average_salary

FROM departments d

JOIN employees e

ON d.department_id = e.department_id

JOIN salaries s

ON e.employee_id = s.employee_id

GROUP BY d.department_name

HAVING AVG(s.amount) > 75000;

MySQL - db_44yowregj

Query 1 x Query 2 x Query 3 x Query 4 x Query 5 x Query 6 x Query 7 x Query 8 x

```

42 GROUP BY d.department_name;
43
44 SELECT
45     d.department_name,
46     AVG(s.amount) AS average_salary
47 FROM departments d
48 JOIN employees e
49     ON d.department_id = e.department_id
50 JOIN salaries s
51     ON e.employee_id = s.employee_id
52 GROUP BY d.department_name
53 HAVING AVG(s.amount) > 75000;

```

department_name	average_salary
Engineering	89250.0000
Analytics	93250.0000
Design	89250.0000
Management	97250.0000
Marketing	83750.0000

CASE EXPRESSIONS

SELECT

CONCAT(e.first_name, ' ', e.last_name) AS emp_name,
s.amount AS salary,

CASE

WHEN s.amount >= 90000 THEN 'High Salary'

WHEN s.amount >= 70000 THEN 'Medium Salary'

ELSE 'Low Salary'

END AS salary_category

FROM employees e

JOIN salaries s

ON e.employee_id = s.employee_id;

MySQL - db_44yurwregj			
EXPLORER	Query 1	Query 2	Query 3
Search...	55 SELECT		
	56 CONCAT(e.first_name, ' ', e.last_name) AS emp_name,		
	57 s.amount AS salary,		
	58 CASE		
	59 WHEN s.amount <= 90000 THEN 'High Salary'		
	60 WHEN s.amount >= 70000 THEN 'Medium Salary'		
	61 ELSE 'Low Salary'		
	62 END AS salary_category		
	63 FROM		
	64 employees e		
	65 JOIN		
	66 departments d		
	67 ON e.dept_id = d.department_id		
	68 ORDER BY		
	69 s.amount DESC		
	70		
	71		
	72		
	73		
	74		
	75		
	76		
	77		
	78		
	79		
	80		
	81		
	82		
	83		
	84		
	85		
	86		
	87		
	88		
	89		
	90		
	91		
	92		
	93		
	94		
	95		
	96		
	97		
	98		
	99		
	100		
	101		
	102		
	103		
	104		
	105		
	106		
	107		
	108		
	109		
	110		
	111		
	112		
	113		
	114		
	115		
	116		
	117		
	118		
	119		
	120		
	121		
	122		
	123		
	124		
	125		
	126		
	127		
	128		
	129		
	130		
	131		
	132		
	133		
	134		
	135		
	136		
	137		
	138		
	139		
	140		
	141		
	142		
	143		
	144		
	145		
	146		
	147		
	148		
	149		
	150		
	151		
	152		
	153		
	154		
	155		
	156		
	157		
	158		
	159		
	160		
	161		
	162		
	163		
	164		
	165		
	166		
	167		
	168		
	169		
	170		
	171		
	172		
	173		
	174		
	175		
	176		
	177		
	178		
	179		
	180		
	181		
	182		
	183		
	184		
	185		
	186		
	187		
	188		
	189		
	190		
	191		
	192		
	193		
	194		
	195		
	196		
	197		
	198		
	199		
	200		
	201		
	202		
	203		
	204		
	205		
	206		
	207		
	208		
	209		
	210		
	211		
	212		
	213		
	214		
	215		
	216		
	217		
	218		
	219		
	220		
	221		
	222		
	223		
	224		
	225		
	226		
	227		
	228		
	229		
	230		
	231		
	232		
	233		
	234		
	235		
	236		
	237		
	238		
	239		
	240		
	241		
	242		
	243		
	244		
	245		
	246		
	247		
	248		
	249		
	250		
	251		
	252		
	253		
	254		
	255		
	256		
	257		
	258		
	259		
	260		
	261		
	262		
	263		
	264		
	265		
	266		
	267		
	268		
	269		
	270		
	271		
	272		
	273		
	274		
	275		
	276		
	277		
	278		
	279		
	280		
	281		
	282		
	283		
	284		
	285		
	286		
	287		
	288		
	289		
	290		
	291		
	292		
	293		
	294		
	295		
	296		
	297		
	298		
	299		
	300		
	301		
	302		
	303		
	304		
	305		
	306		
	307		
	308		
	309		
	310		
	311		
	312		
	313		
	314		
	315		
	316		
	317		
	318		
	319		
	320		
	321		
	322		
	323		
	324		
	325		
	326		
	327		
	328		
	329		
	330		
	331		
	332		
	333		
	334		
	335		
	336		
	337		
	338		
	339		
	340		
	341		
	342		
	343		
	344		
	345		
	346		
	347		
	348		
	349		
	350		
	351		
	352		
	353		
	354		
	355		
	356		
	357		
	358		
	359		
	360		
	361		
	362		
	363		
	364		
	365		
	366		
	367		
	368		
	369		
	370		
	371		
	372		
	373		
	374		
	375		
	376		
	377		
	378		
	379		
	380		
	381		
	382		
	383		
	384		
	385		
	386		
	387		
	388		
	389		
	390		
	391		
	392		
	393		
	394		
	395		
	396		
	397		
	398		
	399		
	400		
	401		
	402		
	403		
	404		
	405		
	406		
	407		
	408		
	409		
	410		
	411		
	412		
	413		
	414		
	415		
	416		
	417		
	418		
	419		
	420		
	421		
	422		
	423		
	424		
	425		
	426		
	427		
	428		
	429		
	430		
	431		
	432		
	433		
	434		
	435		
	436		
	437		
	438		
	439		
	440		
	441		
	442		
	443		
	444		
	445		
	446		
	447		
	448		
	449		
	450		
	451		
	452		
	453		
	454		
	455		
	456		
	457		
	458		
	459		
	460		
	461		
	462		
	463		
	464		
	465		
	466		
	467		
	468		
	469		
	470		
	471		
	472		
	473		
	474		
	475		
	476		
	477		
	478		
	479		
	480		
	481		
	482		
	483		
	484		
	485		
	486		
	487		
	488		
	489		
	490		
	491		
	492		
	493		
	494		
	495		
	496		
	497		
	498		
	499		
	500		
	501		
	502		
	503		
	504		
	505		
	506		
	507		
	508		
	509		
	510		
	511		
	512		
	513		
	514		
	515		
	516		
	517		
	518		
	519		
	520		
	521		
	522		
	523		
	524		

ORDER BY p.budget DESC;

MySQL - db_44yurwregj			
EXPLORER	Query 1	Query 2	Query 3
Search...			
TABLES			
course			
department			
department_salary_summary			
departments			
employee_basic			
employee_hierarchy			
employees			
enrollment			
faculty			
leaves			
salaries			
student			
student_backup			
teaches			
FUNCTIONS			
add_numbers			
annual_salary			
	147	148	149
	150	151	152
	153	154	155
	project_name	department_name	budget
	Cybersecurity Framework	Engineering	1100000.00
	Predictive Analytics Platform	Analytics	950000.00
	Cloud Infrastructure	Engineering	800000.00
	Budget Forecasting System	Analytics	650000.00
	Smart Web Portal	Engineering	550000.00
	Online Sales Campaign	Marketing	480000.00
	Customer Insights Study	Marketing	400000.00
	Staff Development Program	Management	300000.00

EMPLOYEES WORKING ON PROJECTS

SELECT

CONCAT(e.first_name, ' ', e.last_name) AS emp_name,

p.project_name,

ep.hours_worked

FROM employee_project ep

JOIN employees e

ON ep.employee_id = e.employee_id

JOIN project p

ON ep.project_id = p.project_id

ORDER BY ep.hours_worked DESC;

MySQL - db_44yurwregj			
EXPLORER	Query 1	Query 2	Query 3
Search...			
TABLES			
course			
department			
department_salary_summary			
departments			
employee_basic			
employee_hierarchy			
employees			
enrollment			
faculty			
leaves			
salaries			
student			
student_backup			
teaches			
FUNCTIONS			
add_numbers			
annual_salary			
calculate_grade			
getAverageSalaryOfDepartment			
square_number			
student_result			
PROCEDURES			
addStudent			
approveOrRejectLeave			
getAllEmployees			
getAllStudents			
getEmployeesByDepartment			
getStudentsByDepartment			
	220	221	222
	223	224	225
	226	227	228
	229	230	231
	emp_name	project_name	hours_worked
	Isabella Davis	Predictive Analytics Platform	160
	Aisha Malik	Cybersecurity Framework	150
	Robert Johnson	Cloud Infrastructure	150
	Maria Garcia	Budget Forecasting System	150
	Naveed Choudhry	Predictive Analytics Platform	140
	Elena Martinez	Budget Forecasting System	140
	Marcus Hill	Cybersecurity Framework	135
	Ahmed Khan	Budget Forecasting System	130
	Michael Brown	Smart Web Portal	130
	Megan Clark	Staff Development Program	120
	Jamal Williams	Customer Insights Study	120
	John Doe	Smart Web Portal	120
	Olivia White	Staff Development Program	110
	Tariq Ali	Online Sales Campaign	110
	Amanda Williams	Cloud Infrastructure	110
	Daniel Miller	Staff Development Program	100
	Sofia Lopez	Online Sales Campaign	100
	Jane Smith	Smart Web Portal	100
	Sara Johnson	Cloud Infrastructure	90